

Jaxer[®] The world's first Ajax server

[Home](#) [Quick Start](#) [Guide](#) [Tutorials](#) [API](#) [Screencasts](#) [Forums](#) [Contribute](#) [Download](#)

Creating Server-side Images with Jaxer

Introduction

Jaxer is built on the same technology as Firefox which means we can take advantage of XUL-like functionality on the server. In this article, we show a small snippet of JavaScript that creates a simple chart and then injects that into an img element as a data URL. Once the page is received by the client, they'll see an inlined chart that was created completely on the server using only Jaxer and JavaScript.

The Code

The following snippet of JavaScript is the high-level view of the creation of an image and the injection of the result into an image tag.

```
01. function draw()
02. {
03.     // grab target image
04.     var img = document.getElementById("chart");
05.
06.     // create canvas
07.     var canvas = document.createElement("canvas");
08.
09.     // set size to match image
10.     canvas.width = img.width;
11.     canvas.height = img.height;
12.
13.     // draw chart
14.     drawChart(canvas);
15.
16.     // create a data url from the canvas
17.     var dataUrl = canvas.toDataURL("image/png", "");
18.
19.     // apply results to the target image
20.     img.src = dataUrl;
21. }
```



- In our sample HTML document (see attached), we make use of the onserverload attribute to call our draw() function once the page has been loaded inside of Jaxer.
- In line 4, we grab our "chart" img element from the HTML document. We will use this element to specify our chart's dimensions and as the final container for our generated chart.
- Next we create a canvas element so we can take advantage of the rich drawing API provided there. We will use this element to create a chart and to convert that chart into a data URL.
- We want our resulting server-side generated image to match the img's dimensions, so we apply those values to the new canvas element in lines 10 and 11.
- The meat of the drawing occurs in the "drawChart" function. For this sample, we keep things simple. A more complex example would most likely use data from a database query. You can see the full details of our drawing code in the attached file.
- Now that we have an image on our canvas element, we need to convert that to a data URL so we can inline the image. Line 17 conveniently creates our URL in one function call and it lets us choose the image format with which to encode the image data. We use PNG encoding here.
- Finally, we apply the URL to the img's src element which will be serialized out to the client

The Result

Below is the image that is created when running this sample. If you view the source of the generated page, you will see the data url that was generated on the server.



Conclusion

Jaxer makes it easy to inline server-side generate images due to the rich functionality provided by the Mozilla code-base. As seen here, developers can make use of the canvas element and its associate 2D drawing context to create images on the fly before a page is sent to the client. There are many interesting examples of the canvas element in use online. Now JavaScript developers can take advantage of that on the server without having to resort to other server-side languages or executables.